



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 7

**Алгоритми за сортиране. Цифрова
сортировка, лексикографско сортиране.
Сортиране чрез вмъкване и чрез размяна.
Библиотечна функция qsort.**

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

Н. УИРТ



Сортиране. Обща информация

Какво е сортиране?

Сортирането е процес, при който последователност (множество) от n елемента $a_1, a_2, \dots, a_n$ се подреждат в определен ред.

Целта на сортирането е по-късно лесно да се извърши **търсене** на зададени елементи в сортираното множество.

Примери: речници, телефонни указатели, библиотечни каталози, складове, азбучни указатели към книги и др.



Сортиране. Обща информация

Какво е сортиране?

Сортирането се прилага за елементи, за които има пълна подредба (линеен ред) - това е **релация R**, при която за всяка двойка елементи a, b от множеството важи **$a R b$** или **$b R a$** . Релацията $\leq$ за цели или реални числа е линеен ред.

Ако в последователността **$a[i] \leq a[i+1]$** - сортиране в **нарастващ** ред;

Ако последователността **$a[i] \geq a[i+1]$** - сортиране в **намаляващ** ред;



Сортиране. Обща информация

Видове сортиране

Сортирането бива **два** вида:

- **Външно** – данните са на външен носител (файл на флашка, CD), при които достъпа до елементите не е пряк (може да е цикличен или последователен);

- **Вътрешно** – данните са в оперативната памет (с пряк достъп) – Вътрешните се делят на **две подгрупи**:

а) **частни алгоритми** – сортират се елементи със стойности от определен тип и в определен обхват;

б) **общи (универсални) алгоритми** – приложими за елементи от произволен тип за които има пълна подредба.

Когато еднаквите елементи не си променят реда при сортирането се казва, че сортирането е устойчиво!



Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) – частен алгоритъм за сортиране.

Прилага се при сортиране на **цели числа в зададен диапазон** (не много голям – да се помещава в свободната ОП).

Бива **два вида**:

- по **ключ**;
- по **ключ и по номер**.



Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) по ключ:

Прилага се при работа с **едномерен масив** от цели числа в диапазона от $K_{\min}$ до $K_{\max}$ с дължина n .

Пример: масив с елементи в диапазона $K_{\min} = 2$ до $K_{\max} = 10$, и $n=8$:

i	1	2	3	4	5	6	7	8
A[i]	7	3	5	10	3	7	2	7

След сортиране масива добива вида:

i	1	2	3	4	5	6	7	8
A[i]	2	3	3	5	7	7	7	10



Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) по ключ:

Описание на алгоритъма:

Използва се **помощен масив P** с индекси $K_{\min}=2$ до $K_{\max}=10$, т.е. индекси на масива P са ключове на масива A .

В началото **P се нулира**. Обхожда се масив A и за всеки негов елемент $k=A[i]$ в масива P се добавя 1 към елемент $P[k]$. След обхождането всеки елемент $P[k]$ ще съдържа броя на елементите в масива A със стойност k . За случая масива P ще има следния вид:

k	2	3	4	5	6	7	8	9	10
P[k]	1	2	0	1	0	3	0	0	1

Следва обхождане на масив P . За всеки елемент $P[k] > 0$ в масив A се записват $P[k]$ на брой елементи със стойност k .



Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) по ключ:

Процедура, реализираща алгоритъма:

```
void bucketsort (int A[i], int n,  $k_{min}$ ,  $k_{max}$ )
```

```
int P[i]; int l, j, k;
```

```
{
```

```
  for ( $i=k_{min}$ ;  $i \leq k_{max}$ ;  $i++$ )
```

```
    P[i] = 0;
```

```
  for ( $j=0$ ;  $j < n$ ;  $j++$ )
```

```
    P[A[j]] = P[A[j]] + 1;
```

```
  j = 1;
```

```
  for ( $i=k_{min}$ ;  $i < k_{max}$ ;  $i++$ )
```

```
    for ( $k=0$ ;  $k < P[i]$ ;  $k++$ )
```

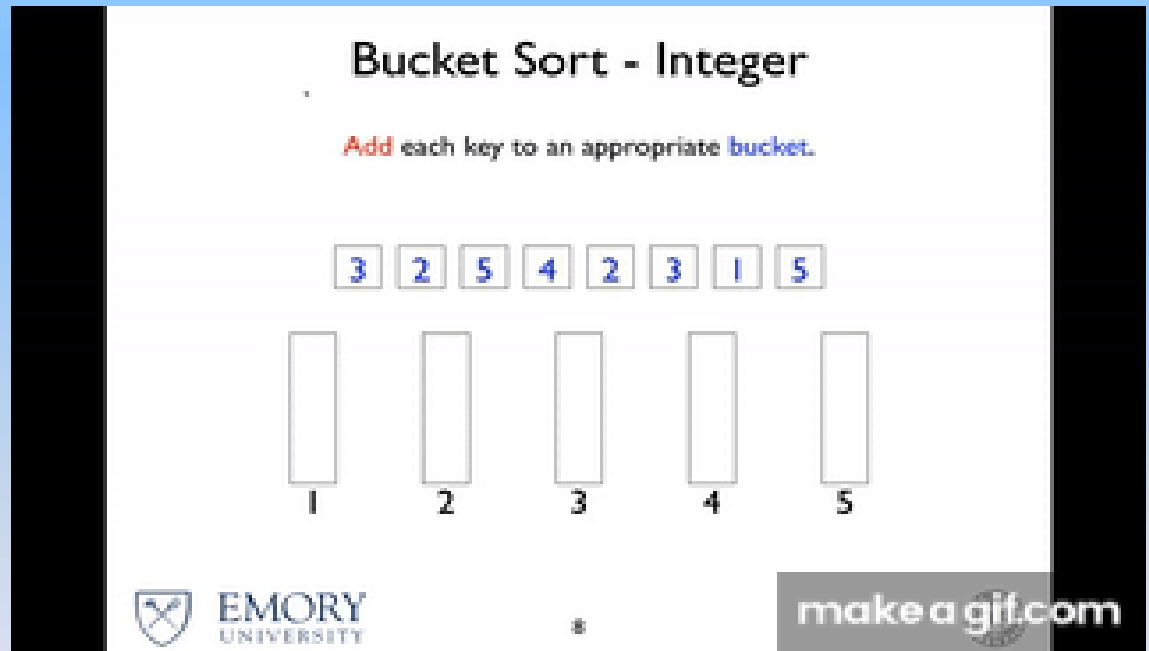
```
    {
```

```
      A[j] = i;
```

```
      j = j + 1;
```

```
    }
```

```
  }
```





Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) по ключ:

Сложността на алгоритъма е:

$$T=O(n+m)$$

където:

n – дължината на масива;

m – дължината на диапазона на целите числа.

Когато $m=O(n)$, тогава $T = O(n)$, и това е **най-бързият** алгоритъм за сортиране в този случай (линеен), т.е. когато диапазона на числата не е голям.



Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) по ключ и по номер:

Описание на алгоритъма:

Тук всеки елемент на **масива A** с **дължина n** е запис с **2 полета: ключ и номер**. Ключът е цяло число в диапазон от **1 до m** , а номерата са от **1 до n** . *Номерата не могат да се повтарят!*

Пример: *Нека стойността на ключа е от 1 до 6 и $n=8$.*

Използва се **нов масив „Начало“** с дължина 1 до m (в случая 6), чийто елемент **$Начало[k]$** е начало на **стек за ключ k** , т.е. **на всяка стойност на ключ k съответства стек**.

Масива **A** се обхожда и номера на всеки негов елемент се записва в стека на неговия ключ. Записването в стека се прави в началото му за да не се губи време за обхождане на **стека в края му**.



Сортиране. Цифрова сортировка

Цифрова сортировка (Bucket Sort) по ключ и по номер:

Следва обхождане на масив *Начало* и четене на стековете, сочени от него и запис на сортираните вече елементи в оригиналния масив. Ако клетката *Начало[k]*, за ключ *k=0* не се извършва четене от стека.

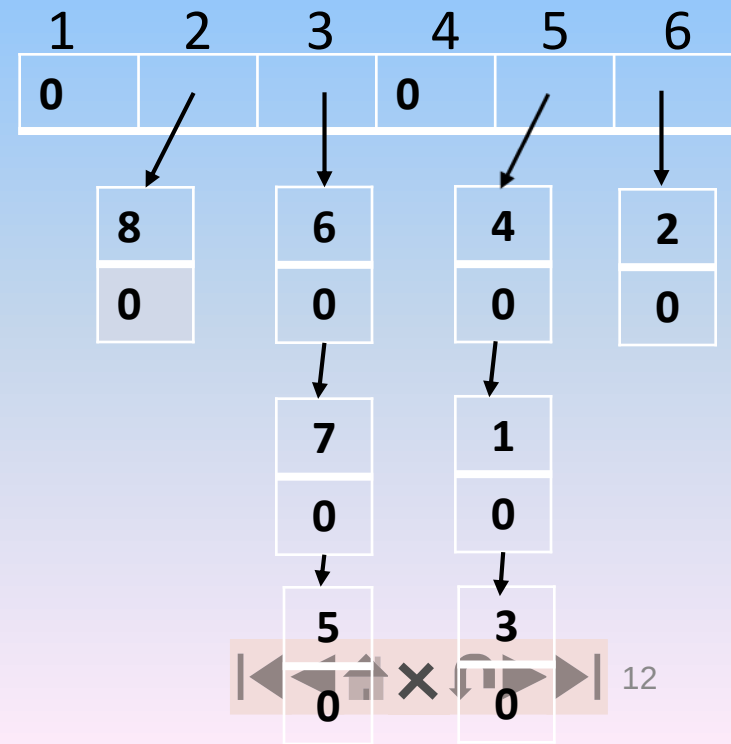
Масив *Начало*:

i	Номер[i]	Ключ[i]
1	5	3
2	3	5
3	8	2
4	7	3
5	1	5
6	6	3
7	2	6
8	4	5

Преди сортиране

i	Номер[i]	Ключ[i]
1	8	2
2	5	3
3	7	3
4	6	3
5	3	5
6	1	5
7	4	5
8	2	6

след сортиране





Сортиране. Лексикографска сортировка

Определение за лексикографски ред:

Нека са дадени две думи:

$X = x_1, x_2, x_3, \dots, x_i, \dots, x_n$

чиито символи са от зададена

$Y = y_1, y_2, y_3, \dots, y_i, \dots, y_m$

азбука.

Тогава:

1) $X=Y$, когато $m=n$, $x_i = y_i$ за $i=1,2,\dots,n$

2) $X<Y$, ако символите от двете думи са равни до i -тия символ, но $x_{i+1} < y_{i+1}$ (символът x_{i+1} е преди символа на y_{i+1} в азбуката), или Y е с по-голяма дължина от X , и X е поддума на Y .

X 



Y 



Първи случай

Втори случай



Сортиране. Лексикографска сортировка

Определение за лексикографски ред:

В лексикографски (азбучен) ред се подреждат думите в речник.

Нека е даден масив от n думи, които трябва да се сортират в лексикографски ред.

Алгоритмите за този вид сортировка са **два варианта:**

- Алгоритми за сортиране на **думи с фиксирана дължина;**
- Алгоритми за сортиране на **думи с променлива дължина.**



Сортиране. Лексикографска сортировка

Алгоритъм за лексикографско сортиране на думи с фиксирана дължина:

Нека е даден масив ***a*** с ***n*** на брой думи, с ***k*** – брой символи във всяка дума и символите са от азбука с ***t*** на брой символи. (***n* = 6, *k* = 4, *t* = 3, азбука *a,b,c***)

Алгоритъм:

Извършват се толкова обхождания на думите, колкото е дължината им (***k***), като при всяко обхождане се извършва сортиране по символа от една и съща позиция на думите.

При 1-то обхождане сортирането е по най-десния символ. Следващите обхождания извършват сортиране съответно по символите от втората, третата и т.н. позиция **отдясно – наляво**. Всяко следващо сортиране запазва наредбата на думите, получени от предходните сортирания.



Сортиране. Лексикографска сортировка

Алгоритъм за лексикографско сортиране на думи с фиксирана дължина:

При всяко обхождане се създават **m – опашки**, всяка от които **съответства на символ**. Опашките са подредени според реда на символите в азбуката. Думата се записва в съответната опашка в зависимост от символа в поредната позиция. При всяко обхождане думите се четат от опашките (**отляво – надясно и отдолу – нагоре**), получени от предишното обхождане и се записват в нови опашки.

Само при първото обхождане думите се четат от масива.

След последното обхождане думите се четат от опашката и се записват в масива.

Сложността на алгоритъма е: **$O((m+n)k)$**



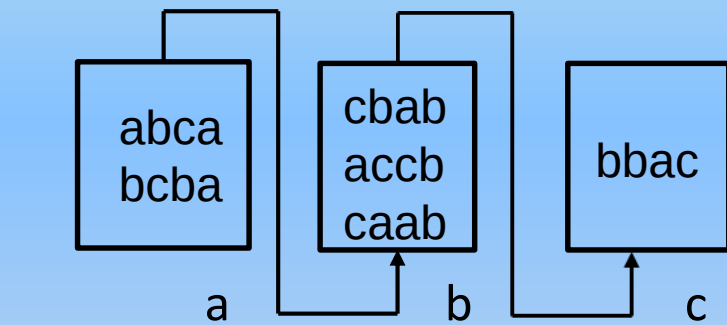


Сортиране. Лексикографска сортировка

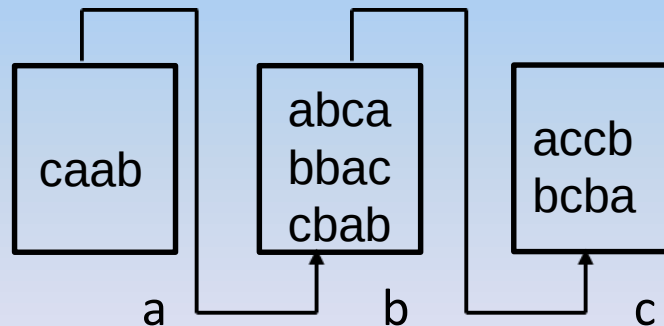
Алгоритъм за лексикографско сортиране на думи с фиксирана дължина:

Пример: $n = 6$, $k = 4$, $t = 3$, азбука a, b, c

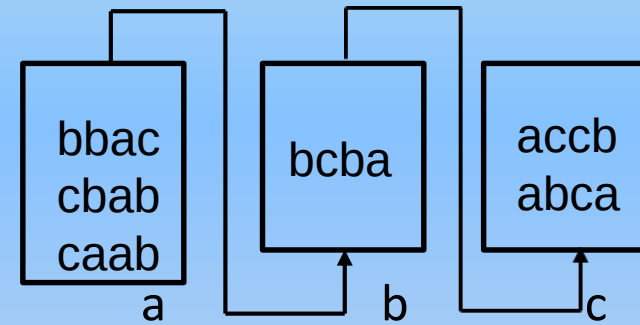
$n=6$	$k=4$	caab
		bcba
		abca
		bbac
		accb
		cbab
		abca
		accb
		bbac
		bcba
		caab
		cbab



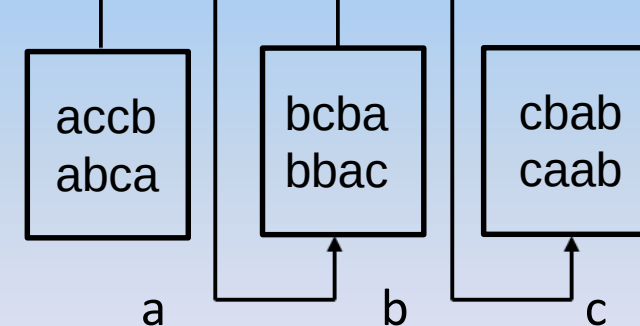
сортиране по 4-я символ



сортиране по 2-я символ



сортиране по 3-я символ



сортиране по 1-я символ

← **сортиран масив**



Сортиране. Лексикографска сортировка

Алгоритъм за лексикографско сортиране на думи с променлива дължина:

Има два варианта:

При **1-ви вариант** – *изравняват се дължините на думите спрямо най-дългата дума*, като по-късите думи се допълват открай с празни символи (шпация). След това се прилага описания преди алгоритъм. Този алгоритъм е неефективен, ако в масива има само няколко дълги думи – правят се излишни обхождания на думи с еднакъв празен символ в дадена позиция.

При **2-ри вариант** (по-ефективен) – първо *думите се сортират* с цифрова сортировка в намаляващ ред на дължините им. След това се обхождат от най-дясна позиция ($k=1$). При k -тото обхождане в сортирането участват само думите, които имат символ в позиция k (отдясно - наляво), т.е думите с дължина:

$$L_{\max} - k + 1$$



Сортиране. Лексикографска сортировка

Пример: сортиране на 10 думи с помощта на 2 масива от стрингове:

```
#include<stdio.h>
#include <string.h>
int main()
{
    int i,j;
    char str[10][50],temp[50];
    printf("Enter 10 words:\n");
    for(i=0;i<10;++i)
        gets(str[i]);
    for(i=0;i<9;++i)
        for(j=i+1;j<10 ;++j)
        {
```





Сортиране. Лексикографска сортировка

```
if(strcmp(str[i],str[j])>0)
{
    strcpy(temp,str[i]);
    strcpy(str[i],str[j]);
    strcpy(str[j],temp);
}
}
printf("In lexicographical order: \n");
for(i=0;i<10;++i){
    puts(str[i]);
}
return 0;
}
```



Сортиране. Основни групи

Най-общо могат да се обобщят следните **групи методи за сортиране**:

- Сортиране чрез **вмъкване**;
- Сортиране чрез **размяна**;
- Сортиране чрез **избор**;
- Сортиране чрез **сливане**;
- Сортиране чрез **разпределение**.

Ще разгледаме няколко алгоритъма от изброените групи методи и тяхната реализация.



Сортиране. Основни групи

За оценка на ефективност на един алгоритъм за сортиране се използват следните означения: **С** - брой на сравненията на ключовете и **М** - брой на разместванията на елементите. Те се явяват **функция на n** - брой на елементите, които се подреждат.

Простите и очевидните методи имат сложност **$O(n^2)$** ; по-ефективните - **$O(n * \text{Log}(n))$** .



Сортиране. Сортиране чрез вмъкване

Сортиране чрез просто вмъкване:

Към тази група се отнасят методите: сортиране чрез **просто вмъкване**, сортиране чрез **двоично вмъкване**, сортиране по метода на **Шел**, сортиране чрез **вмъкване в списък**, сортиране чрез **изчисляване на адреса** и др.

1. Сортиране чрез просто вмъкване (Straight insertion sort)

Идеята на метода е проста: **масивът** от подрежданите елементи се дели на две части, **първата** част съдържа **елементите от $A[1]$ до $A[i-1]$** и вече е подредена, **втората** част - **само елемента $A[i]$** , който трябва да се включи към вече подредения масив.

6 5 3 1 8 7 2 4



Сортиране. Сортиране чрез вмъкване

Сортиране чрез просто вмъкване:

Следват две примерни рекурсивни реализации на метода.

- **Включване чрез полуобмен:**

```
void ins_p(int a[], int i)
{
    int j; int c=a[i];
    j=i-1;
    while ((j>=0)&&(a[j]>c))
    {
        a[j+1]=a[j--];
    }
    a[j+1]=c;
    if (i<num-1) ins_p(a, i+1);
}
```




Сортиране. Сортиране чрез вмъкване

Сортиране чрез просто вмъкване:

- Включване чрез разместване:

```
void ins_r(int a[], int i)
{
    int rab, j;
    j=i-1;
    while ((j>=0)&&(a[j]>a[j+1]))
    {
        rab=a[j];
        a[j]=a[j+1];
        a[j+1]=rab;
        j--;
    }
    if (i<num-1)
        ins_r(a, i+1);
}
```



Сортиране. Сортиране чрез вмъкване

Сортиране чрез просто вмъкване:

Методът е **устойчив**.

Ефективен е, когато масивът е частично подреден. **Препоръчва се при сравнително малки n .**

Сложността на метода се оценява като **$O(n^2)$.**

Insertion	Минимален	Среден	Максимален
С (бр. сравнения)	$n-1$	$(n^2+n-2)/4$	$(n^2+n)/2-1$
М (бр. размени)	$2(n-1)$	$(n^2+9n-10)/4$	$(n^2+3n-4)/2$

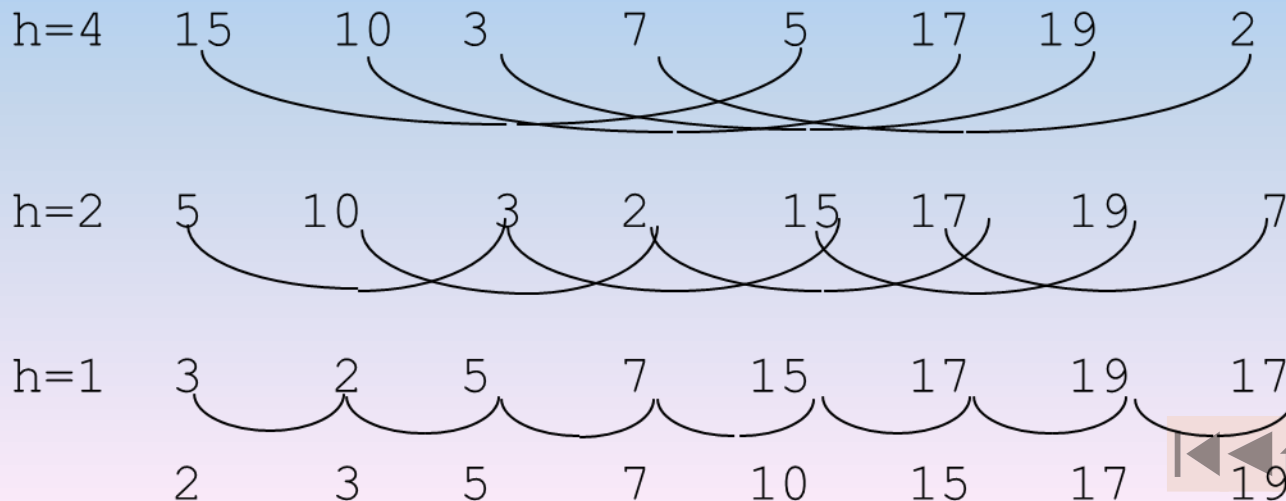


Сортиране. Сортиране чрез вмъкване

Сортиране по метод на Шел:

2. Сортиране по метода на Шел (Shell sort)

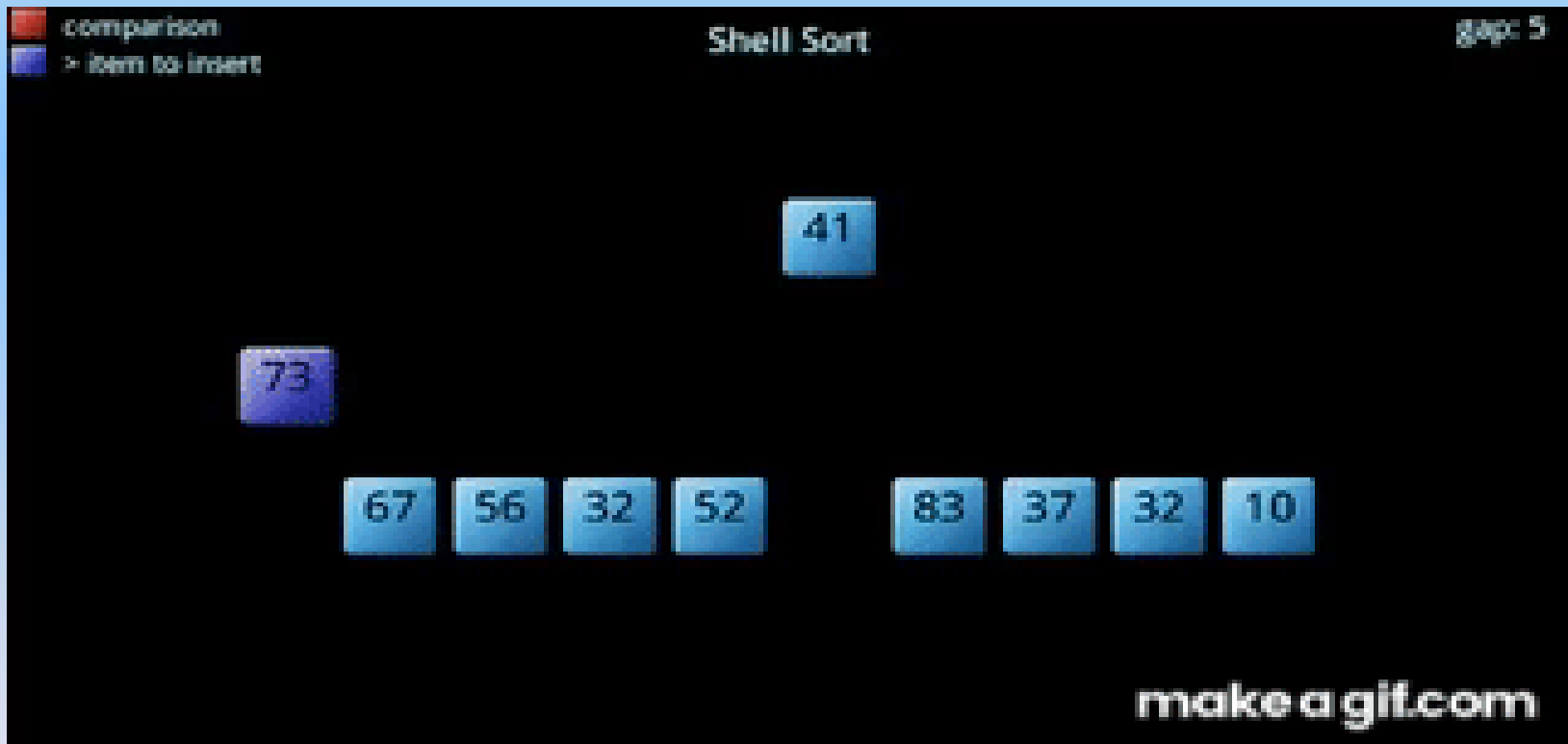
Методът е предложен през 1959 г. от Д. Шел и носи името на своя автор. Шел **подобрява** метода на просто вмъкване **чрез групиране на подрежданите елементи**, които се намират на **разстояние h** един от друг. Всяка такава група се нарича **серия** и се подрежда отделно. След това **стъпката h** се намалява и сортирането се повтаря. Процесът продължава до получаване на **$h < 1$** .





Сортиране. Сортиране чрез вмъкване

Сортиране по метод на Шел:





Сортиране. Сортиране чрез вмъкване

Сортиране по метод на Шел:

Препоръчва се началната стъпка да има стойност приблизително $n/9$.

Стъпките образуват следната поредица: $h_{k-1}=3h_k+1$

D.Knuth препоръчва: 1 4 13 40 121 364 1093 3280 9841 и т.н.

```
void shellsort(int a[], int left, int right)  //left - най-малкия индекс на масива
                                              //right - най-големия индекс на масива
{  int h;                                     //стъпка
  for (h=1; h<=(right-left)/9; h=3*h+1)  //изчисляване на h
    for(;h>0;h/=3)                        //сортиране по серии със стъпка h
      for (int i=left+h; i<=right; i++)
      {
        int j=i; int v=a[i];
        while (j>=left+h && v<a[j-h])
          {a[j]=a[j-h];j-=h;}
        a[j]=v; }
}
```

Методът е **неустойчив**. Сложността на алгоритъма се оценява като

$O(n * \text{Log}(n))$. Препоръчва се при сравнително големи масиви.



Сортиране. Сортиране чрез размяна

Сортиране - метод на мехурчето:

Към тази група се отнасят: метод на мехурчето, усъвършенстван метод на мехурчето, бързата сортировка, обменна поразредена сортировка и др.

1. Сортиране чрез пряка размяна или метод на мехурчето (Bubble sort).

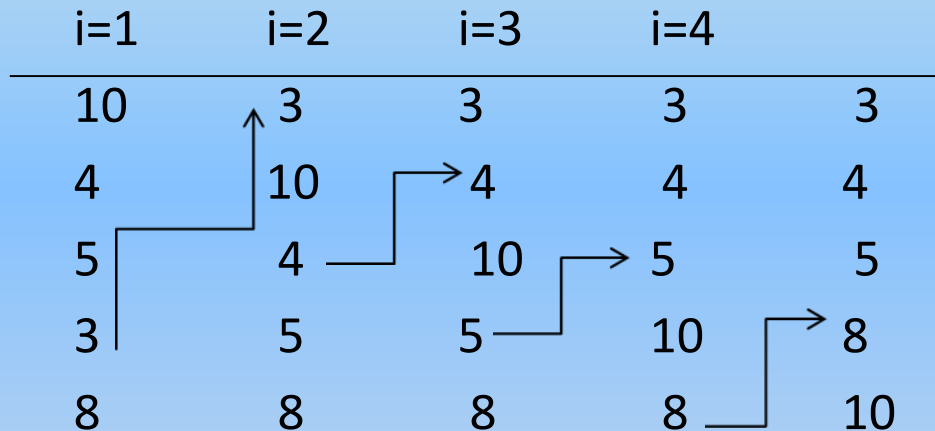
Идеята на метода се състои в следното: сравняват се и при необходимост се размятат всеки два съседни елемента. Названието на метода идва от аналогията с въздушни мехурчета - при последователни преглеждания на масива с данни “по-леките” (с по-малък ключ) елементи постепенно “изплуват” нагоре - в началото на масива.



Сортиране. Сортиране чрез размяна

Сортиране - метод на мехурчето:

АКО $x_{j-1} > x_j$, ТО x_j И x_{j-1} РАЗМЕНЯТ МЕСТАТА СИ:



```
void Bubblesort1(int a[], int n)
```

```
{ int r, i, j;
```

```
  for (i=0; i<n-1; i++)
```

```
    for (j=n-1; j>=i; j--)
```

```
      if (a[j-1]>a[j])
```

```
        {r=a[j];
```

```
          a[j]=a[j-1];
```

```
          a[j-1]=r; } }
```

6 5 3 1 8 7 2 4



Сортиране. Сортиране чрез размяна

Сортиране – метод на мехурчето:

Сортиране по метода на мехурчето – 2 вариант:

```
void Bubblesort2(int a[], int i)
{int inv, r;
  do
  { inv=0;                                //флаг за инверсия или необходимост от размяна
    for (int j=0; j<i-1; j++)
      if (a[j]>a[j+1])
      {
        inv=1;
        r=a[j];
        a[j]=a[j+1];
        a[j+1]=r;
      }
    }
  while (inv);
}
```




Сортиране. Сортиране чрез размяна

Сортиране – метод на мехурчето:

Методът има **сложност $O(n^2)$** . Препоръчва се при сравнително малки n . Методът е подобрен чрез въвеждането на допълнителна логическа променлива, която следи за наличието на инверсии, изискващи размяна на съседните елементи в масива. Ако при поредния преглед не е открита нито една инверсия, алгоритъмът завършва своята работа.

Основните характеристики на сортирането по метода на мехурчето са:

Bubblesort	Минимален	Среден	Максимален
С (бр. сравнения)	$n(n-1)/2$	$n(n-1)/2$	$n(n-1)/2$
М (бр. размени)	0	$3(n^2-n)/4$	$3(n^2-n)/2$



Сортиране. Сортиране чрез размяна

Сортиране – метод на мехурчето:

Методът може да бъде подобрен още веднъж по следния начин:

безсмислено е да преглеждаме елементите, за които със сигурност се знае, че са на окончателните си места. Сортирането се извършва **в обратна посока**: започваме **от “дъното”** в края на масива и **най-тежките елементи потъват към него**. В променливата **k** ще съхраняваме максималния индекс, при който е била извършена размяна на предходната итерация. Елементите, разположени вдясно от последната размяна, са на окончателните си места.

```
void Bubblesort3(int a[], int n)  
{int r, i, j, k;  
  for (i=n; i>0; i=k)  
    for (k=j=0; j<i; j++)  
      if (a[j]>a[j+1])  
        {r=a[j]; a[j]=a[j+1]; a[j+1]=r;  
        k=j;}  
}
```



Сортиране. Сортиране чрез размяна

Сортиране чрез клатене:

2. Сортиране чрез клатене

Въпреки ниската си ефективност метода на мехурчето дава възможност за редица подобрения. Разгледахме две от тях (чрез добавяне на флаг и поддържане на променлива, указваща максималния индекс на размяна от предходната стъпка).

Методът има една особеност: отделен лек елемент в края на масива ще изплува нагоре за една-единствена итерация, докато един тежък елемент, разположен близо до повърхността, ще потъва надолу само с една позиция при всяка стъпка на алгоритъма. Ако обърнем посоката на циклите (както сме направили във втория алгоритъм) ситуацията ще се обърне, но асиметрията ще се запази. Идеята на предлагания подход е посоката на сортиране да се обръща на всяка стъпка. Това е трето подобрение на алгоритъма. **Прилагането на трите подобрения едновременно дава нов алгоритъм - сортиране чрез клатене.**



Сортиране. Сортиране чрез размяна

Сортиране чрез клатене:

```
void shakersort (int a[], int n)  
{  
    unsigned k=n, R=n-1;  
    unsigned L=1, j, rab;  
    do  
    {  
        for (j=R; j>=L; j--)  
            if (a[j-1]>a[j])  
                {rab=a[j]; a[j]=a[j-1]; a[j-1]=rab; k=j;}  
        L=k+1;  
        for (j=L; j<=R; j++)  
            if (a[j-1]>a[j])  
                {rab=a[j]; a[j]=a[j-1]; a[j-1]=rab;k=j;}  
        R=k-1;  
    }  
    while (L<=R); }
```





Сортиране. Сортиране чрез размяна

Сортиране чрез клатене:

В най-добрия случай горния алгоритъм извършва **$n-1$** сравнения, а в средния случай - **n^2** . Макар и да има известно подобрене в сравнение с класическия метод на мехурчето, общата ефективност на алгоритъма не се подобрява съществено, т. к. сортирането чрез клатене не намалява броя на извършваните размени. А размяната по-принцип е неколkokратно по-тежка операция от сравнението.

Основният **недостатък** на метода е в “**микроскопичния**” му подход - **винаги се сравняват два съседни елемента**. Този недостатък е отстранен в следващия разгледан метод.



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

3. Сортиране чрез сегментиране или бързо сортиране, или сортиране на Хоар (Quicksort)

Целта на предстоящото подобрене на основната идея е да се минимизира броя на размените в масива. От последното подобрене (или сортиране чрез клатене) стана ясно, че той ще бъде редуциран, ако разменяме първия елемент с последния, втория - с предпоследния и т. н., сортирайки целия масив за $[n/2]$ размени.

Изводът: колкото на по-големи разстояния се извършват размените, толкова е по-голяма тяхната ефективност.



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Идеята на алгоритъма, предложен от Хоар, е да **изберем** някакъв елемент **X** и да **разделим** масива на две части: **лява**, в която елементите **са по-малки от X**, и **дясна**, в която са **по-големи**. Положението на **X** (т. е. индекса му в масива) след преобразуването **се фиксира и повече не се променя**.

Прилагаме същия алгоритъм за лявата и дясната части, **стеснявайки постепенно границите на разглежданите подмасиви**, докато не достигнем до **интервали (сегменти)**, съдържащи **единствен елемент** (оттук и названието - сортиране чрез сегментиране).

След приключване на работата на алгоритъма масивът е подреден.



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Да означим с ***q*** индекса на ***x*** в масива, т.е ***x = a[q]***.

$a[i] \leq x$		x	$a[i] \geq x$	
0	q-1	q	q+1	n-1

Нека сортираме подмасива ***a [L, L+1,..., R]*** и нека ***partition()*** разделя подмасива на две части, извършва в него съответните преобразувания и връща като резултат ***q*** (но не търси в масива ***X***, а го определя по определени правила! Как?). Най-общо бързото сортиране може да се опише така:

```
void quicksort (int L, int R)
{ int q;
  if (L < R)
  { q=partition(L, R);
    quicksort(L, q);
    quicksort(q+1, R); } }
```




Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Как да извършим разделянето на масива?

Има **два различни подхода** или два варианта на *partition()*.

Вариант 1: избира се елемент x , по който се извършва разделянето. В лявата част на масива трябва да останат елементи, по-малки или равни на x , а в дясната - по-големи от x . Текущия дял от масива $a[L, L+1, \dots, R]$ се разглежда отляво надясно, като при това в лявата му част се изгражда постоянно разширяваща се област от елементи, по-малки или равни на x . Десният край на областта се определя от q . Когато j достигне края R на дяла, q ще сочи границата между двете области.

unsigned partition1 (int L, int R)

```
{ int q, j, x, rab;
```

```
  q=L-1; x=a[R];
```

```
  for (j=L; j<=R, j++)
```

```
    if (a[j]<=x) { rab=a[q]; a[q]=a[j]; a[j]=rab;
```

```
      q++; }
```

```
  if (j==R)  q--; return q;
```

```
}
```

6 5 3 1 8 7 2 4



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Вариант 2: използват се два индекса - j и i , показващи границите на две непрекъснато разширяващи се области от двата края към центъра. На всяка стъпка на алгоритъма се прави опит за разширяване на лявата област надясно, докато това е възможно, т. е. докато вдясно от нея стои елемент, $\leq$ от x . Същото се извършва и за дясната област (двата *while*-цикъла). След това се разменят местата на двата елемента-прегради за лявата и дясната области, които са спрели разширяването им, и процесът се повтаря. Приключва при “срещане” на двете области, т. е. когато двете им граници се разминат. Индексът i сочи десния край на лявата област, а j - левия край на дясната

unsigned partition2 (int L, int R)

{ int i, j, x, rab;

i=L; j=R; x=a[L];

do { while (x>a[i]) i++;

while (x<a[j]) j--;

if (i<=j) {rab=a[i]; a[i]=a[j]; a[j]=rab; i++; j--; } }

while (i <=j); return j; }

Unsorted Array





Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

За сортиране на двете части на масива се извършват **две последователни рекурсивни обръщания**. Т. к. второто винаги се отлага (двете обръщания не могат да се извършат едновременно!), то може да се реализира **итеративно - чрез цикъл**.

Възможно е алгоритъмът изцяло да бъде реализиран итеративно - **чрез използване на** специално създаден за целта **стек**. На всяка стъпка лявата част на масива ще се обработва директно, докато дясната част ще се поставя в стека, като обработката ѝ ще се отлага до момента, когато се приключи окончателно с лявата част и всички свързани с нея подзадачи.



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Остава открит въпроса: как да се избира средния елемент - **x**?

Най-добре на всяка стъпка (при всяко сегментиране) да се избира **средния по големина** елемент. Тогава **общия брой сравнения** ще бъде **$n * \log_2 n$** .

Очакваният **брой размени** при всяко разделяне е от порядъка на **$n/6$** . Оттук **броят на размените в оптималния случай** е:

$$(n/6) * n * \log_2 n$$

Вероятността да улучим средния елемент е **$1/n$** , но **средната ефективност** на бързото сортиране не зависи само от **n** .



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Все пак, как да избираме елемента X , относно който разделяме масива?

Един възможен подход е да вземаме винаги фиксиран елемент от масива, например първия, последния или средния (с индекс $[n/2]$). Изборът на средния елемент е изключително важен за скоростта на алгоритъма: колкото по-близък е избрания елемент до средния *по големина* елемент за дяла, толкова по-равномерно става разбиването на масива на две части, а оттук - по-малка дълбочина на рекурсията, т. е. по-ефективна работа на алгоритъма. Можем ли да избираме за разделящ елемент медианата на два или повече елемента? Да, но това е свързано с допълнителни операции и води до забавяне, което не винаги се компенсира с извлечените ползи и не води задължително до подобрене.



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Съществуват и други фактори, влияещи на ефективността на алгоритъма:

- силата на бързото сортиране е в неговата “**хазартност**”. Колкото по-разбъркан е масивът, толкова по-голяма е ефективността на метода. Хоар препоръчва изборът на x да се прави по случаен начин или като средно аритметично на три или повече случайно избрани елемента;
- алгоритъмът е ефективен само при големи n . Затова при получаване на сегмент с големина ≤ 15 елемента се препоръчва да се премине към някой друг алгоритъм и сортирането да се довърши чрез използване на друг алгоритъм. Подмасивът е почти сортиран и е добър вход за сортирането чрез вмъкване: вмъкванията ще се извършват в рамките на съответната област, т. е. гарантирано на малки разстояния



Сортиране. Сортиране чрез размяна

Бързо сортиране (на Хоар):

Съществуват и други фактори, влияещи на ефективността на алгоритъма:

- силата на бързото сортиране е в неговата “**хазартност**”. Колкото по-разбъркан е масивът, толкова по-голяма е ефективността на метода. Хоар препоръчва изборът на x да се прави по случаен начин или като средно аритметично на три или повече случайно избрани елемента;
- алгоритъмът е ефективен само при големи n . Затова при получаване на сегмент с големина ≤ 15 елемента се препоръчва да се премине към някой друг алгоритъм и сортирането да се довърши чрез използване на друг алгоритъм. Подмасивът е почти сортиран и е добър вход за сортирането чрез вмъкване: вмъкванията ще се извършват в рамките на съответната област, т. е. гарантирано на малки разстояния

Сортиране. Библиотечна функция *qsort*

Езикът C/ C++ има библиотечна функция *qsort*, която може да сортира произволен масив:

```
# include <stdlib.h>
```

```
...
```

```
qsort(a, n, sizeof(T), compare);
```

където *a* е подреждания масив, *n* е броят на елементи в него, *T* е типа на елементите в масива, *compare* е наша собствена функция, показваща как трябва да бъдат сравнявани два елемента *a[i]* и *a[j]* от масива.

Тази функция е подобна на добре известната функция *strcmp*, която връща 0 при равенство, някакво отрицателно число при “по-малко” и някакво положително число при “по-голямо”.



Сортиране. Библиотечна функция *qsort*

Пример: сортиране на масив от цели числа с библиотечната функция *qsort*:

```
#include <iostream.h>
#include <stdlib.h>
const n=100;
int a[n], num;
void input(int a[])
{
    cout<<"\n Input num<100:";
    cin>>num; cout<<"\n"<<num;
    cout<<"\n Input items:\n";
    for (int i=0; i<num; i++)
    {
        cout<<"=";
        cin>>a[i];
    }
}
```



Сортиране. Библиотечна функция *qsort*

```
void out(int a[])
{
    cout<<"\n Array:\n";
    for (int i=0;i<num; i++)
        cout<<a[i]<<" ";
}

int compare (const void *p, const void *q)
{
    return *(int*)p < *(int*)q ? -1: *(int*)p > *(int*)q;
}

void main()
{
    input(a);
    out(a);
    qsort(a, num, sizeof(int), compare);
    out(a);
}
```